

Automatic Test Case Generation vs. AI-Assisted Testing: A Benchmark of Botsing, CATG, Deepseek, and Gemini

จัดทำโดย

663380507-9 นางสาวนิภาดา ญายะนันท์	Section 1
673380584-2 นางสาวกัตติพิชา วะสาร	Section 2
673380602-6 นางสาวศศิวิตร วังษ์รุ่งอรุณเลิศ	Section 2

เสนอ

ผศ.ดร.ชิตสุรา สุ่มเล็ก

ที่มาและความสำคัญ

- ▶ การสร้าง Unit Test Case แบบ Manual ใช้เวลาและทรัพยากร โดยเฉพาะซอฟต์แวร์ที่มีความซับซ้อน
- ▶ จึงมีแนวทาง Automatic Test Case Generation (ATCG) เพื่อสร้าง Test Case โดยอัตโนมัติ
- ▶ ปัจจุบัน Generative AI สามารถช่วยวิเคราะห์ Source Code และสร้าง Test Case ผ่าน Prompt ได้
- ▶ โครงการนี้จึงเปรียบเทียบ 2 แนวทาง

✦ **ATCG:** Botsing, CATG

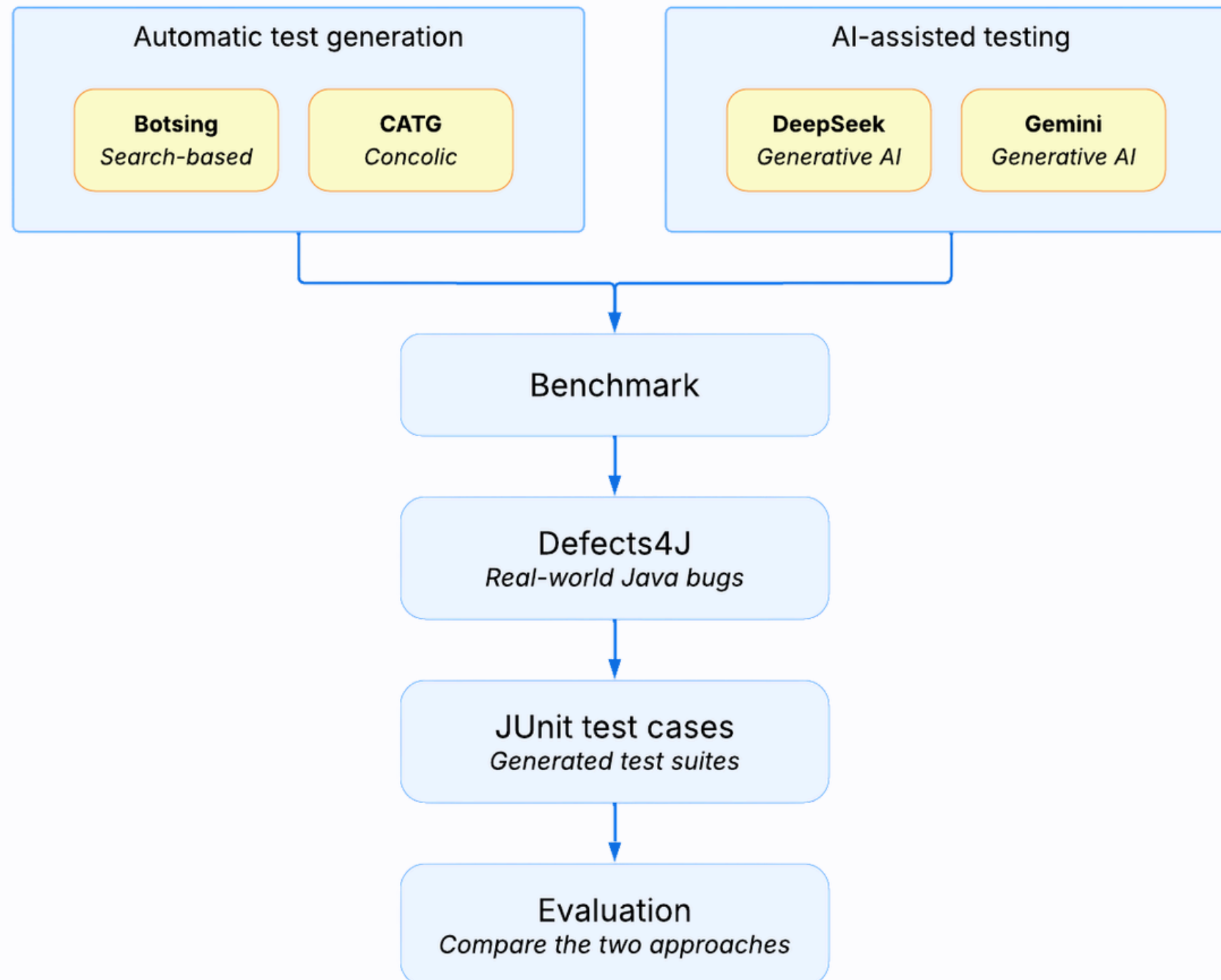


✦ **AI-Assisted Testing:** DeepSeek, Gemini

วัตถุประสงค์

- 1 ศึกษา Botsing และ CATG สำหรับการสร้าง Test Case อัตโนมัติ
- 2 ประยุกต์ใช้ DeepSeek และ Gemini และออกแบบ Prompt สำหรับการสร้าง Test Case
- 3 ประเมิน คุณภาพและประสิทธิภาพ ของ Test Case ที่สร้างขึ้น
- 4 เปรียบเทียบ ATCG vs. AI-Assisted Testing และวิเคราะห์ข้อดี ข้อจำกัด และความเหมาะสม

แนวคิดการเปรียบเทียบ



เปรียบเทียบ 2 แนวทางในการสร้าง JUnit Test Case

ATCG: Botsing และ CATG

AI-Assisted Testing: DeepSeek และ Gemini

ทั้ง 4 เครื่องมือทดสอบกับ Defects4J และนำ Test Case ที่สร้างได้เข้าสู่การ Evaluation ด้วยเกณฑ์เดียวกัน

Defects4J

- ชุดข้อมูลข้อผิดพลาดจริงจากโครงการ Open Source ภาษา Java
- รวม 17 โครงการ
- แต่ละ Bug มี
 - Buggy Version เวอร์ชันที่มีข้อผิดพลาด
 - Fixed Version เวอร์ชันที่แก้ไขแล้ว
- จัดทำ Bug Manifest สำหรับระบุข้อมูลที่ใช้ในการทดลอง
 - Project / Bug ID
 - Target Class
 - Target Method

Automatic Test Case Generation (ATCG)



Botsing

Botsing

Search-Based Test Generation

- ใช้เทคนิค Search-Based Software Testing เพื่อสร้าง JUnit Test Case
- ค้นหา Test Case ที่ช่วยเพิ่ม Code Coverage
- ใช้ Genetic Algorithm ในการค้นหาชุด Test Case ที่เหมาะสม



CATG

CATG

Concolic Test Generation

- ใช้เทคนิค Concolic Testing ในการสร้าง Test Input
- ผสานการทำงานของ Concrete Execution และ Symbolic Execution
- ใช้ Constraint Solver เพื่อค้นหา Input ที่ทำให้โปรแกรมเข้าสู่เส้นทางใหม่

AI-Assisted Testing



"Gemini"
gemini-3.7-flash



"DeepSeek"
deepseek-v4-pro

เลือกใช้โมเดล เชื่อมต่อผ่านบริการ KKU API เพื่อทำหน้าที่เป็น
เครื่องมือสร้างชุดการทดสอบอัตโนมัติ

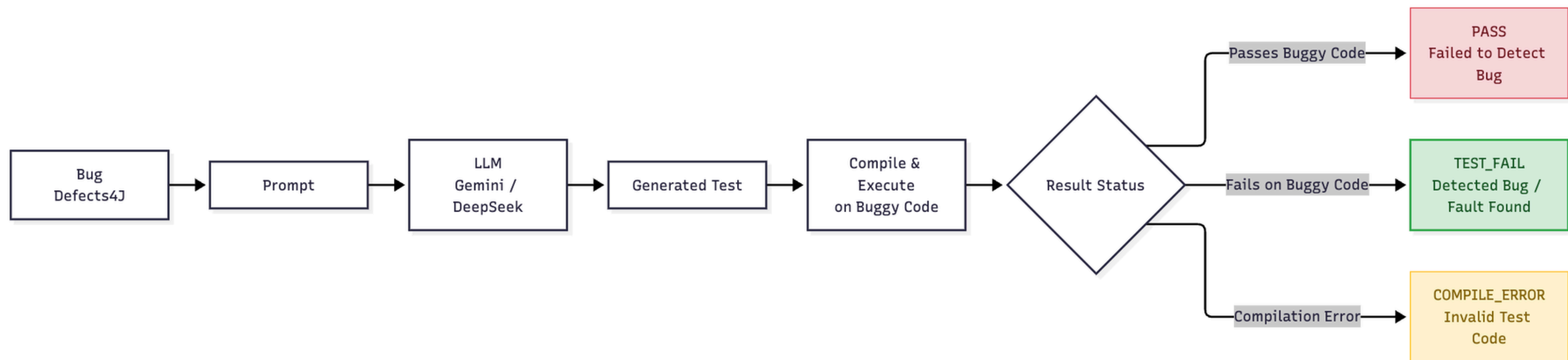
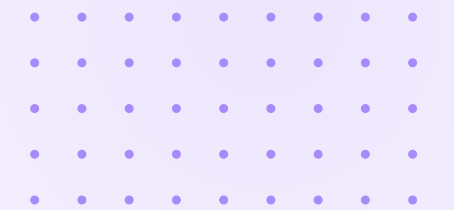
Model Configuration

- Temperature = 0.2 กำหนดค่าเท่ากัน เพื่อให้การทดลองควบคุมปัจจัยแปรผันและเปรียบเทียบผลได้อย่างเป็นธรรม
- Max Tokens = 8,192 กำหนดขีดจำกัดเอาต์พุตสูงสุดเท่ากัน เพื่อรองรับการสร้างคลาสทดสอบขนาดใหญ่และซับซ้อนได้

ที่มาและเป้าหมาย

ใช้ LLM (Gemini, DeepSeek) สร้างเทสอัตโนมัติ เทียบกับ CATG บนชุดข้อมูล Defects4J v3.0.1 (17 โปรแกรม) วัดสถานะการรัน การตรวจจับ Fault และประสิทธิภาพ

วิธีทดลอง



Prompt Design

โมเดลสวมบทบาทเป็น Software Quality Assurance Engineer ที่มีความเชี่ยวชาญสูงด้าน Java Unit Testing ด้วย JUnit 4 และการวิเคราะห์ Benchmark Defects4J กำหนดเป้าหมายให้วิเคราะห์ซอร์สโค้ด Java ที่แนบไป และสร้างชุดการทดสอบ JUnit 4 ที่มีความครอบคลุมสูง

You are an expert Software Quality Assurance Engineer specializing in Java Unit Testing with JUnit 4 and Defects4J benchmark analysis.
Sasiwitra (งาน)

Your task is to analyze the attached Java source file and generate a comprehensive, high-coverage JUnit 4 test suite.

STRICT CONSTRAINTS & REQUIREMENTS:

1. Target Framework & Compatibility:

- Use JUnit 4 ONLY (org.junit.*). Do NOT use JUnit 5/Jupiter.
- Use standard JUnit 4 assertions (org.junit.Assert.*).
- Ensure compatibility with Java 8 (JDK 1.8).

2. Package & Class Naming:

- Extract and use the EXACT package declaration defined in the attached Java file.
- Name the test class as <TargetClassName>Test (e.g., if target is FastMath.java, name it FastMathTest).

3. Testing Focus:

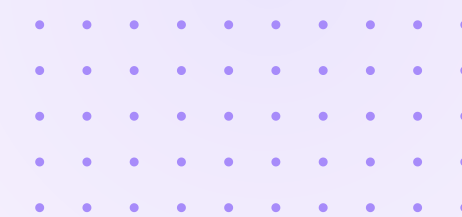
- Target maximum Line Coverage and Branch Coverage on the provided source code.
- Implement deep assertions to detect logical faults and boundary errors (not just null checks).
- Handle edge cases, null arguments, arithmetic boundaries, and complex branching paths.
- For expected exceptions, use @Test(expected = ...) or try-catch blocks with Assert.fail().
- Keep the suite compact: each @Test method must be short and focused.

4. OUTPUT FORMAT (MANDATORY):

- Output ONLY the raw executable Java source code.
- Do NOT wrap code in markdown tags (do NOT use ```java or ```).
- Do NOT include any conversational text, greetings, explanations, or notes before or after the code.
- Do NOT write any comments or Javadoc inside the code (no //, no /* */).
- Do NOT describe or summarize the tests. Do NOT think out loud.
- Respond immediately with the code. Keep reasoning minimal.
- The very first line of your response MUST be the "package ..." statement.

Fault Detection

Gemini 66.67% เทียบกับ DeepSeek 45.87% ของรายการที่รับได้
เห็นว่าโปรเจกต์ไหนจับได้ดีหรือไม่ดี
(Gemini: Chart ต่ำ 3.85%, Codec/Csv/Gson 100%)



ประสิทธิภาพ

เวลาเฉลี่ยต่อ Bug 29.07 วินาที เทียบกับ 108.65 วินาที (เร็วกว่าราว 3.7 เท่า)
และจำนวนเทสเฉลี่ย 26.21 เทียบกับ 57.10 ตัวต่อ Bug

**DeepSeek สร้างเทสมากกว่าแต่ผลไม่ดีกว่า
เพราะโค้ดถูกตัดจบ (Token Limit) จนคอมไพล์ไม่ผ่าน**

เปรียบเทียบประสิทธิภาพของ Gemini และ DeepSeek

ตัวชี้วัด	Gemini	DeepSeek
อัตราคอมไพล์ผ่าน (ของรายการที่รันได้)	99.21%	57.26%
เวลาเฉลี่ยต่อ Bug (วินาที)	29.07	108.65
เวลาเฉลี่ยต่อ Test Case (วินาที)	1.11	1.90
Test Case เฉลี่ยต่อ Bug (ตัว)	26.21	57.10
เวลารวม (ชั่วโมง)	2.80	2.90
จำนวน Bug ที่ใช้คำนวณเวลา	347	96

ข้อจำกัดของการทดลอง

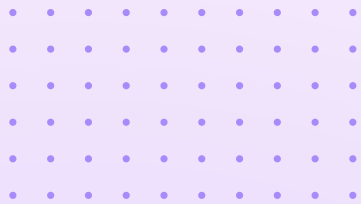
- DeepSeek รับได้ 351 จาก 854 Bug เพราะข้อจำกัด API Keys
- Gemini รับได้ 851 จาก 854 Bug เพราะข้อจำกัด API Keys
- ยังไม่ได้วัด Coverage
- ข้อมูลเวลาและจำนวนเทสมีเฉพาะบางส่วนของ Bug

สรุปและข้อเสนอ

ข้อมูลที่มี Gemini ได้ผลดีกว่าทุกด้าน และงานต่อไปคือรับ DeepSeek ให้ครบ วัด Line/Branch Coverage และเทียบบนชุด Bug เดียวกัน

Experimental Configuration

Dataset	Defects4J
Programming Language	Java
ATCG Tools	Botsing, CATG
AI Tools	Gemini, Deepseek
Build Tool	Maven
Java Version	Java 8 (JDK 1.8)
Operating System	MacOS, Windows
จำนวน Bug	854 Bugs จาก Defects4J
จำนวนรอบการทดลอง	1 รอบต่อ Bug (โดยกำหนดให้แก้ไขโค้ดซ้ำได้สูงสุด 3 รอบ/Attempt ในกรณีที่ทดสอบไม่ผ่าน)
Timeout	กำหนดเวลาในการสร้าง Test Case ต่อ Bug
Input	โปรแกรม Java ที่มี Bug จาก Defects4J
Output	Test Cases ที่แต่ละเครื่องมือสร้างขึ้น



Experimental Procedure

(ขั้นตอนการดำเนินงาน)

ขั้นตอน	รายละเอียด
เตรียม Dataset	เลือกโปรเจกต์และ Bug จาก Defects4J
เตรียมสภาพแวดล้อม	ติดตั้ง Java, Maven และเครื่องมือที่ใช้
เตรียมโปรแกรมที่มี Bug	Checkout Bug version ของ Defects4J
สร้าง Test Case	ใช้เครื่องมือ ATCG ได้แก่ Botsing, CATG และ AI ได้แก่ Gemini, Deepseek
รัน Test Case	นำ Test Case ที่สร้างได้ไปทดสอบกับโปรแกรม
เก็บผลลัพธ์	บันทึกจำนวน Test Case, เวลา, Test Case ที่ผ่าน/ไม่ผ่าน และข้อผิดพลาด
เปรียบเทียบผล	เปรียบเทียบผลระหว่างแต่ละเครื่องมือ
วิเคราะห์ผล	วิเคราะห์ประสิทธิภาพและข้อจำกัดของแต่ละวิธี

Botsing: Experimental Setup



- **Dataset:** Defects4J — Commons-Lang
- **Scope:** 5 Bugs (Lang 1b, 2b, 3b, 4b, 5b)
- **Target Frame:** Frame 5 (Fixed Baseline สำหรับรัน Batch)
- **Exception Types:** หลากหลายประเภทตาม Stack Trace จริงของแต่ละ Bug (เช่น NumberFormatException, NullPointerException)

Frame 1

Frame 2

Frame 3

Frame 4

Frame 5

org.apache.commons.lang3.math.NumberUtils.createInteger

- **Algorithm:** Guided Genetic Algorithm (GGA)
- **Execution Mode:** Automated Batch Processing (run_batch_botsing.sh)
- **Search Budget:** 60 วินาที / Bug
- **Environment:** Dual Java Setup
 - Java 11 (Defects4J Checkout/Compile)
 - Java 8 + JUnit 4.12 (Botsing Runtime)

Botsing: ผลการทดลอง

Bug ID	Exception Type	Target Frame	Status	Result / Cause
Lang 1b	NumberFormatException	Frame 5	SUCCESS	Reproduced สำเร็จ (สร้าง JUnit @Test ได้)
Lang 2b	NullPointerException	Frame 5	FAILED	Stack Trace Depth ไม่ตรงกับ Frame 5
Lang 3b	NumberFormatException	Frame 5	FAILED	Stack Trace Depth ไม่ตรงกับ Frame 5
Lang 4b	StringIndexOutOfBoundsException	Frame 5	FAILED	Stack Trace Depth ไม่ตรงกับ Frame 5
Lang 5b	IllegalArgumentException	Frame 5	FAILED	Java 11 Module Prefix (java.base/) ใน Log
Lang 1b	NumberFormatException	Frame 5	SUCCESS	Reproduced สำเร็จ (สร้าง JUnit @Test ได้)
Lang 2b	NullPointerException	Frame 5	FAILED	Stack Trace Depth ไม่ตรงกับ Frame 5

- ✨ **Pipeline Success:** สร้าง Automation Script (run_batch_botsing.sh) รับประมวลผลต่อเนื่องครบ ทั้ง 5 Bugs สำเร็จ 100%
- **Reproduction Rate:** 20% (1/5 Bugs) ภายใต้เงื่อนไขการตั้งค่า Fixed Target Frame 5
- **Verified Case:** Lang 1b ทำซ้ำข้อบกพร่อง (Crash) สำเร็จ และผ่านการยืนยันบน JUnit 4

```
at eu.stamp.botsing.reproduction.CrashRe
at eu.stamp.botsing.Botsing.parseCommand
at eu.stamp.botsing.Botsing.main(Botsing
>>> Finished Lang 5b <<<

=== All Bugs Completed! ===
```

Botsing: วิเคราะห์ผล

✨ ผลที่พบ ✅

- Automated Execution: Pipeline สามารถทำ Checkout, Compile, Clean Log และรัน Botsing อัตโนมัติโดยไม่สะดุด
- High Precision on Matched Frame: Lang 1b สำเร็จ 100% เนื่องจาก Frame 5 ชี้ตรงไปยังคลาส NumberUtils ของ Application
- Fast Execution Time: ใช้เวลาดำเนินการทั้ง Pipeline เพียง ~7.5 นาที (เฉลี่ย 1.5 นาที / Bug)

✨ ข้อจำกัด ⚠️

- Static Target Frame Limitation: การตั้งค่า - target_frame 5 แบบคงที่ ไม่รองรับทุก Bug เพราะ Call Stack ของแต่ละ Bug มีความลึกไม่เท่ากัน
- Java Cross-Version Parsing: Log จาก Java 11 มี Prefix โมดูล java.base/ ติดมา ทำให้ Botsing (Java 8) อ่าน Class ไม่เจอ
- Input Sensitivity: Botsing ต้องการ Pure Stack Trace จึงต้องตัดบรรทัดหัวข้อ --- ออกจาก Log ก่อนประมวลผล

จากการทดสอบ 5 Bugs พบว่า Botsing มีประสิทธิภาพสูงมากเมื่อระบุ Target Frame ตรงจุด จึงสรุปได้ว่าระบบ Auto-reproduction Pipeline ในอนาคตจำเป็นต้องใช้อัลกอริทึมค้นหา Target Frame แบบไดนามิก (Dynamic Target Frame Detection)

Botsing: Generated Test Case Structure

การวิเคราะห์โครงสร้าง JUNIT TEST CASE ที่สร้างโดย BOTSING (CRASH REPRODUCTION)

```
/*
 * This file was automatically generated by EvoSuite
 * Wed Sep 30 01:28:30 ICT 2026
 */
package org.apache.commons.lang3.math;

import org.junit.Test;
import static org.junit.Assert.*;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS = true, use
public class NumberUtils_ESTest extends NumberUtils_ESTest_scaffolding {
    @Test
    public void notGeneratedAnyTest() {
        // EvoSuite did not generate any tests
    }
}
```

TARGET FRAME GUIDANCE (FRAME 5):

พิสูจน์ว่าเมื่อระบุ TARGET FRAME (FRAME 5) ตรงกับจุดเกิด CRASH ใน APPLICATION CODE (NUMBERUTILS) BOTSING สามารถค้นหา INPUT เพื่อกระตุ้น CRASH ได้ถูกต้อง

GUIDED GENETIC ALGORITHM (GGA):

อัลกอริทึมปรับแต่งพารามิเตอร์ INPUT จนเกิด NUMBERFORMATEXCEPTION ตรงตาม CRASH LOG จริง

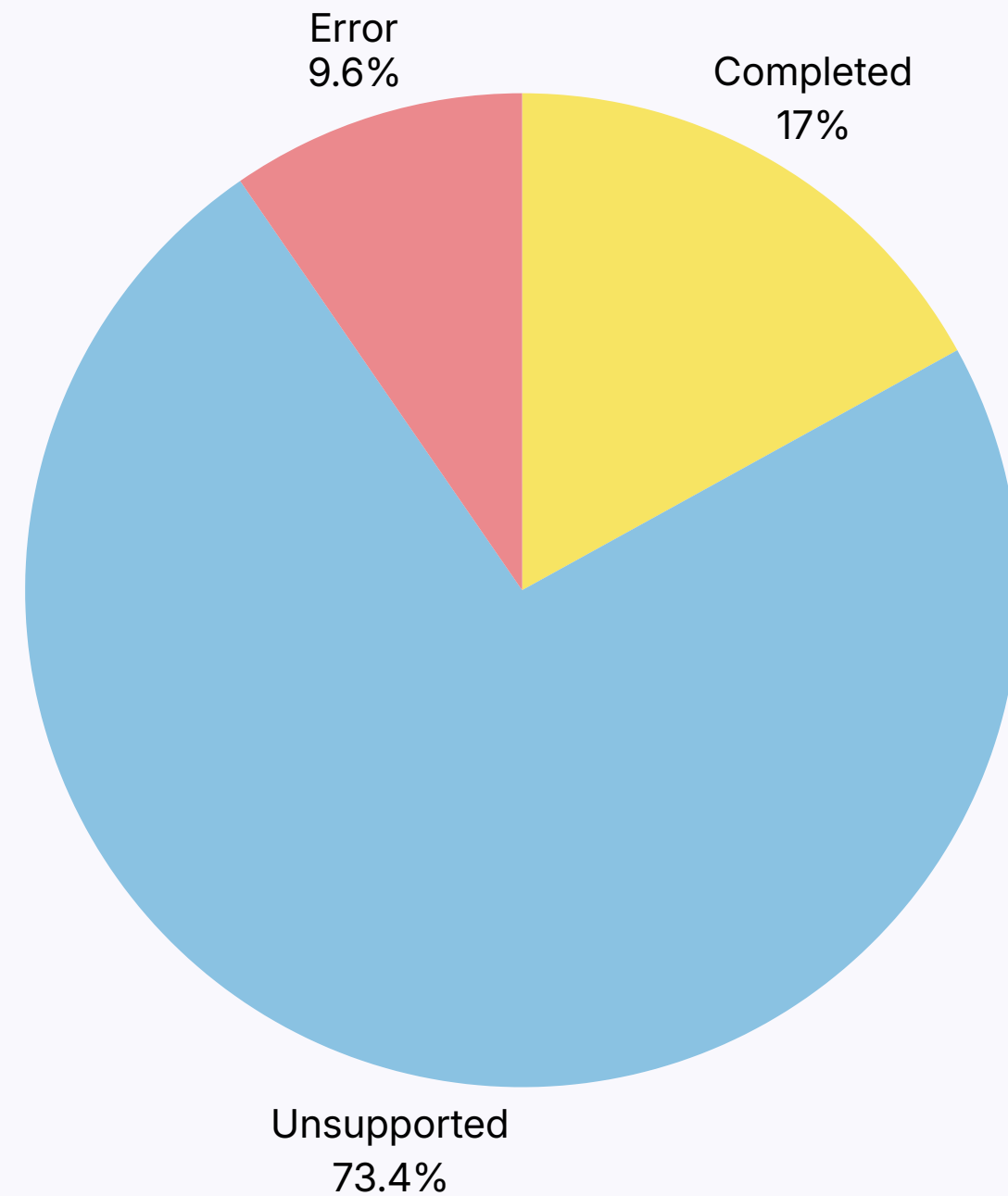
EXECUTABLE & VALIDATED:

ได้โค้ด @TEST บน JUNIT 4 ที่นำไปคอมไพล์และรันทำซ้ำอาการพังได้จริง 100%

CATG: Experimental Setup

รายการ	รายละเอียด
ชุดข้อมูล	Defects4J v3.0.1
จำนวน Bug	854 Active Bugs
จำนวนโปรเจกต์	17 โปรเจกต์
เครื่องมือ	CATG
เทคนิค	Concolic Testing (Concrete + Symbolic Execution)
Solver	CVC4 v1.8
การทดลองต่อ Bug	1 ครั้ง
Search Budget	Default = 5 Concolic Iterations / Bug
Version ที่ทดสอบ	Buggy Version และ Fixed Version
ตัวชี้วัด	Test Case Count, Fault Detection, Line Coverage, Branch Coverage, Generation Time
ไฟล์ผลลัพธ์	benchmark_results.csv

CATG: ผลสถานะการประมวลผล



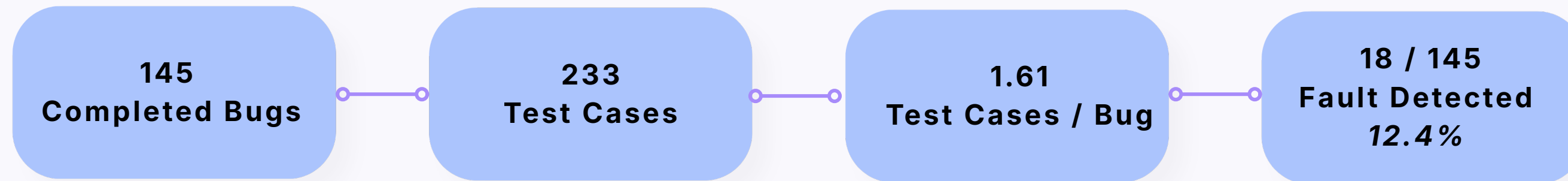
CATG — Processing Status

145 / 854 Bugs (17.0%) สามารถประมวลผลและนำผลไปประเมินได้

- Completed: 145 Bugs (17.0%)
- Unsupported: 627 Bugs (73.4%)
- Error: 82 Bugs (9.6%)

ผลการทดลองส่วนใหญ่ไม่สามารถประเมินได้ เนื่องจากข้อจำกัดของ Test Harness และปัญหาในกระบวนการเตรียม/ประมวลผล ไม่ควรตีความทั้งหมดว่าเป็นข้อจำกัดของ CATG โดยตรง

CATG: Test Case & Fault Detection



Test Case Generation

- สร้าง Test Case รวม 233 Cases
- เฉลี่ย 1.61 Cases / Bug
- จำนวนต่อ Bug: 1 หรือ 5 Cases



Fault Detection

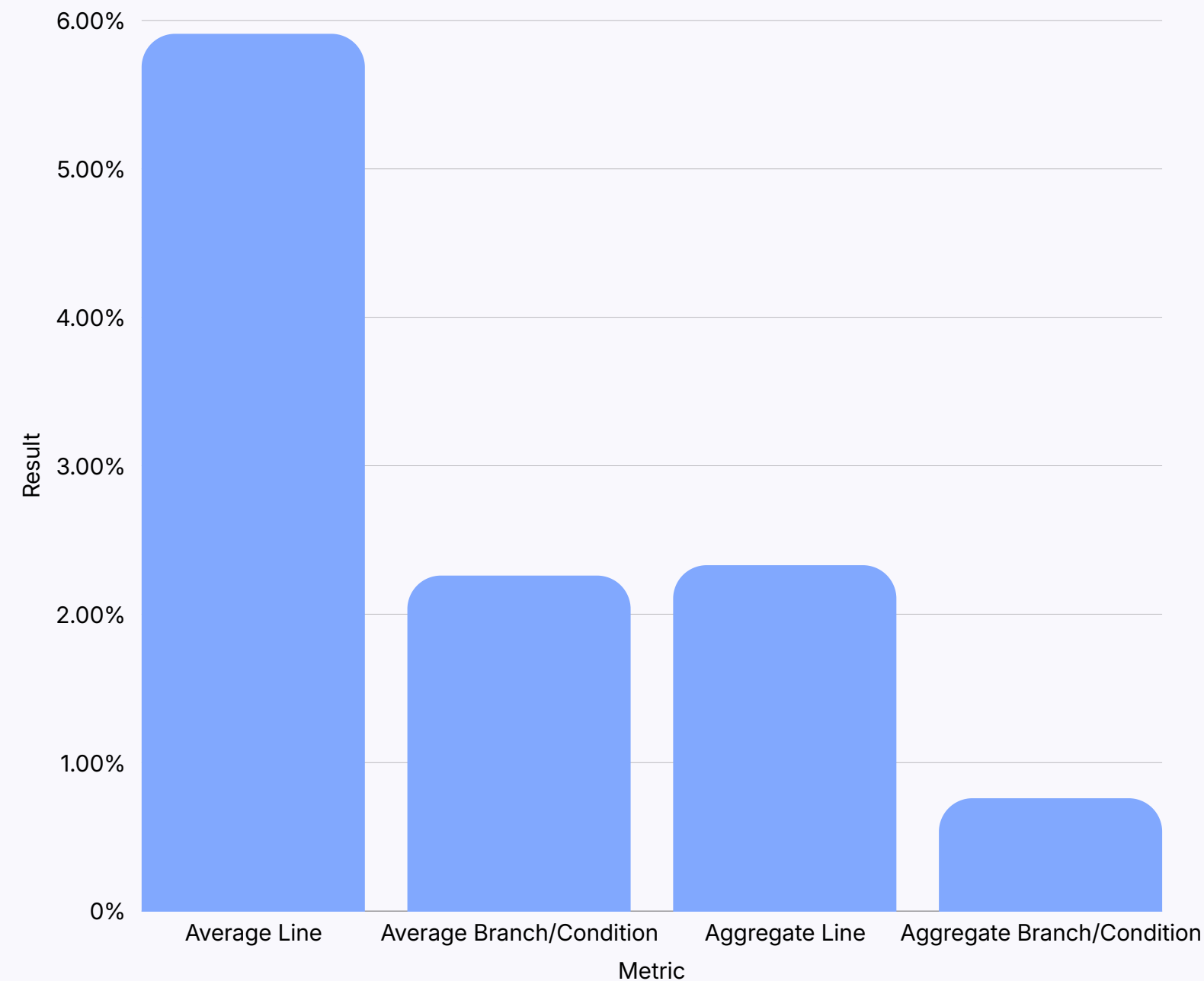
- ตรวจพบ Fault: 18 / 145 Bugs
- Fault Detection Rate: 12.4%
- Not Detected: 127 Bugs (87.6%)



เกณฑ์ Detected:

Buggy Version → Failure
Fixed Version → Pass

CATG: Coverage & Generation Time



- Average: ค่า Coverage เฉลี่ยของแต่ละ Bug
- Aggregate: Coverage รวมจากจำนวน Lines/Branches ที่ถูก Cover เทียบกับทั้งหมด

CATG มี Coverage ค่อนข้างต่ำ โดยค่าเฉลี่ยต่อ Bug อยู่ที่ 5.91% Line และ 2.26% Branch/Condition

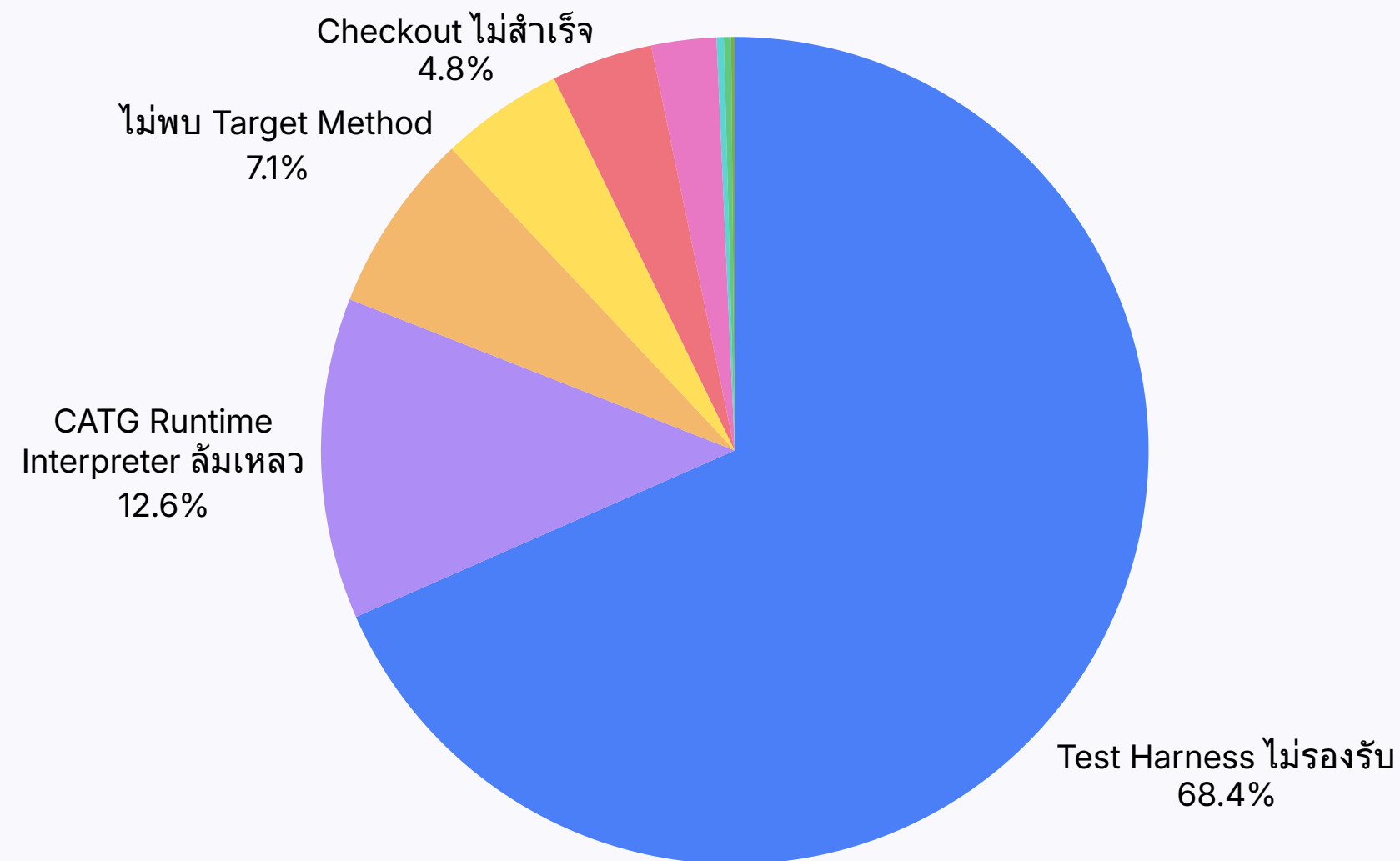


Generation Time

- Mean 2.64 sec / Bug
- Median 1.89 sec / Bug
- P95 6.73 sec
- Min-Max 1.24–21.17 sec

CATG ใช้เวลาในการสร้าง Test Case เฉลี่ย 2.64 วินาที ต่อ Bug โดยค่ามัธยฐานอยู่ที่ 1.89 วินาที และ 95% ของ Bugs ใช้เวลาไม่เกิน 6.73 วินาที

CATG: Analysis of Unsupported & Error



ผลการประเมิน

- 709 / 854 Bugs (83.0%) ไม่สามารถประเมินผลได้

สาเหตุหลัก

- Test Harness: 485 Bugs (68.4%)
- CATG Runtime Interpreter: 89 Bugs (12.5%)
- No Target Method: 50 Bugs (7.1%)
- Checkout ไม่สำเร็จ: 34 Bugs (4.8%)
- Compile ไม่สำเร็จ: 28 Bugs (3.9%)
- CATG Exit ผิดปกติ: 18 Bugs (2.5%)
- Coverage / Test Runner / Source Patch: 5 Bugs (0.7%)

ข้อจำกัดของ Test Harness เป็นสาเหตุหลัก คิดเป็น 485 Bugs (68.4%) ของ Bugs ที่ไม่สำเร็จ

ข้อสังเกต

ผลการทดลองที่ไม่สำเร็จ ไม่ได้เกิดจากข้อจำกัดของ CATG เพียงอย่างเดียว แต่ ยังได้รับผลกระทบจาก Test Harness, Target Method และสภาพแวดล้อม ของ Defects4J

Test Harness

คือ โค้ดตัวกลางที่ใช้ เตรียม Input / Object และเรียก Target Method เพื่อ เชื่อมระหว่าง Defects4J กับ CATG

ข้อจำกัดที่พบ

- รองรับ Parameter Type บางประเภทไม่ได้
- บาง Target Method เป็น Abstract Class / Interface
- บางกรณีไม่สามารถสร้าง Object หรือ Runner สำหรับเรียก Method ได้

ภาพรวมของเครื่องมือทั้งสี่

ด้าน	Botsing	CATG	DeepSeek	Gemini
ประเภท	Search-based (Guided GA)	Concolic Testing (CVC4)	LLM (AI-Assisting)	LLM (AI-Assisting)
เป้าหมาย	ทำซ้ำ Crash จาก Stack Trace	สำรวจเส้นทางโปรแกรม	สร้างชุดทดสอบจากพรอมต์	สร้างชุดทดสอบจากพรอมต์
ขอบเขตที่ทดลอง	1 บั๊ก (Lang_1b)	854 บั๊ก	854 บั๊ก (รันได้ 351)	851 บั๊ก (รันได้ 762)
Input ที่ต้องใช้	Stack Trace + Target Frame	Target Method ที่รองรับชนิดพารามิเตอร์	ข้อมูลบั๊กในพรอมต์	ข้อมูลบั๊กในพรอมต์
Test Case ต่อ Bug	1	เฉลี่ย 1.61	เฉลี่ย 57.10	เฉลี่ย 26.21
เวลาต่อ Bug	30–61 วินาที	เฉลี่ย 2.64 วินาที	เฉลี่ย 108.65 วินาที	เฉลี่ย 29.07 วินาที

เปรียบเทียบตามตัวชี้วัด

✦ อัตราการประมวลผลสำเร็จ

- Botsing: สำเร็จ 100% แต่ทดสอบเพียง 1 บั๊ก จึงสรุปเป็นภาพรวมไม่ได้
- CATG: Completed เพียง 145/854 (17.0%) ส่วน Unsupported 73.4% และ Error 9.6%
- DeepSeek: รับผิดชอบเพียง 351/854 (41.1%) เพราะข้อจำกัด API Key และข้อผิดพลาดระหว่างรัน
- Gemini: รับผิดชอบ 762/851 (ประมาณ 89.5%) สูงที่สุดในกลุ่มที่ทดลองทั้งชุดข้อมูล

✦ อัตราคอมไพล์ผ่าน (ของรายการที่รับผิดชอบ)

- Gemini 99.21% (COMPILE_ERROR เพียง 6 รายการ) สูงกว่า DeepSeek 57.26% มาก
- DeepSeek มี COMPILE_ERROR ถึง 42.7% ของรายการที่รับผิดชอบ สาเหตุหลักคือ Token Limit Truncation ทำให้โค้ดถูกตัดก่อนจบ

เปรียบเทียบตามตัวชี้วัด



การตรวจจับ Fault

- Botsing: TRUE (Lang_1b)
- CATG: 18/145 (12.4%) ใช้เกณฑ์เข้มงวด คือ Fail บน Buggy และ Pass บน Fixed
- DeepSeek: TEST_FAIL 161 รายการ (45.87% ของที่รับได้)
- Gemini: TEST_FAIL 508 รายการ (66.67% ของที่รับได้)



Code Coverage

- มีเฉพาะ CATG: Line Coverage เฉลี่ย 5.91% และ Branch 2.26% (Aggregate 2.33% และ 0.76%)
- Gemini และ DeepSeek ยังไม่ได้วัดเนื่องจากข้อจำกัดด้านเวลา

วิเคราะห์ผลลัพธ์การทดลอง

เครื่องมือ	ผลเด่น	ข้อจำกัด / ข้อสังเกต
Botsing	ตรวจจับ Fault ได้แม่นยำ เมื่อมี Stack Trace + Target Frame	ใช้งานเฉพาะทาง และต้องมี Stack Trace
CATG	สร้าง Test Input ได้เร็ว เฉลี่ย 2.64 วินาที/Bug	Success 17.0% ได้รับผลกระทบจาก Test Harness และ Environment; Coverage ต่ำ
Gemini	ครอบคลุม Dataset กว้าง, Compile 99.21%, Fault Detection 66.67% ของที่รันได้	ผลต่างกันตามโปรเจกต์ เช่น Chart ต่ำกว่าโปรเจกต์อื่นมาก
DeepSeek	สร้าง Test Case ต่อ Bug ได้มากกว่า Gemini (57.10 vs 26.21)	Test Case จำนวนมาก Compile ไม่ผ่าน, ใช้เวลามากกว่า ~3.7 เท่า, ประมวลผลได้ 41.1%

ประเด็นปัญหาที่พบ

กลุ่มปัญหา	รายละเอียด	ผลกระทบ
Test Harness (CATG)	ชนิดพารามิเตอร์ไม่รองรับ (461), ส่งค่า null ให้ String ไม่ได้, Unsafe.allocateInstance() ทำให้ Field เป็น null, Abstract Class (24)	68.4% ของบักที่ไม่สำเร็จ
ตัวเครื่องมือ CATG	ClassCastException (67), ArrayIndexOutOfBoundsException (22), LinkageError, NoSuchMethodException	Error 89 + 18 บัก
สภาพแวดล้อม/Defects4J	Checkout patch ไม่สำเร็จ (34), Compile ไม่สำเร็จ (28; encoding ของ Math, Gradle ของ Mockito, build ของ Jsoup), Cobertura instrumentation ล้มเหลว (2)	64 บัก
การเก็บผลลัพธ์	.strip() ตัดช่องว่าง, Identity Hash ไม่คงที่	Fault Detection คลาดเคลื่อน 4 รายการ
LLM (DeepSeek)	Token Limit Truncation, API Key จำกัด	COMPILE_ERROR 42.7% และรันได้เพียง 41.1%
LLM ทั้งสอง	ไม่ได้ตรวจ Buggy/Fixed, ไม่ได้วัด Coverage, ข้อมูลระดับเมรอตมีเพียงบางส่วน	เปรียบเทียบเชิงคุณภาพไม่ได้เต็มที่
Botsing	ทดลองเพียง 1 บัก, ต้องมี Stack Trace	ขาดการสรุปเชิงสถิติ

ข้อเสนอแนะและแนวทางการพัฒนาต่อ

- 1 **ปรับปรุง Harness ของ CATG** (รองรับชนิดข้อมูลเพิ่ม, ส่ง null ได้, สร้าง Object ด้วย Constructor จริง, เก็บ Expected Outcome โดยไม่ .strip())
- 2 **ตรวจ Fault Detection ของ Gemini และ DeepSeek** ด้วยเกณฑ์เดียวกับ CATG (Fail บว Buggy และ Pass บว Fixed)
- 3 **วัด Line/Branch Coverage ของ AI-Assisted** ให้ครบ และเปรียบเทียบบบชุด Bug เดียวกัน (Intersection)
- 4 **แก้ปัญหา Token Limit** ของ DeepSeek ด้วยการแบ่งพรอมต์ตามเมธอด และรันให้ครบ 854 บั๊ก
- 5 **ขยายการทดลอง Botsing** ไปยังบั๊กประเภท Crash หลาย ๆ รายการใน Defects4J เพื่อให้ได้ข้อมูลเชิงสถิติ
- 6 **พิจารณาแนวทางผสม** เช่น ใช้ Botsing กับบั๊ก Crash, CATG กับเมธอดที่มีเงื่อนไขซับซ้อน และ AI-Assisted สำหรับสร้างชุดทดสอบกว้าง ๆ แล้วใช้เครื่องมือเชิงอัลกอริทึมเสริมความครอบคลุม